

PHÂN TÍCH QUICK GUIDE

QUY TRÌNH XỬ LÝ TÀI LIỆU CHO NLCS RAG

Bản hướng dẫn triển khai chi tiết: việc cần làm, thủ công/tool, công nghệ và cách tối ưu

Ngày tạo: 23/05/2026

1. Tóm tắt mục đích của Quick Guide

Quick Guide mô tả toàn bộ pipeline xử lý tài liệu cho hệ thống RAG: từ file gốc PDF/DOCX/Image, chuyển sang Markdown chuẩn, gắn metadata, kiểm tra hiệu lực, review, publish, sau đó backend mới chunk, embedding và index vào Qdrant.

Ý chính cần nhớ: PDF/DOCX/Image gốc chỉ là nguồn đối chiếu; Markdown trong 01_Dataset mới là nguồn chính để chunk và embedding. Backend chỉ được ingest tài liệu đã đủ điều kiện: OCR xong, review approved, hiệu lực valid, rag_status published và confidentiality public.

Điều kiện publish tối thiểu:

- ocr_status: "done"
- review_status: "approved"
- validity_status: "valid"
- rag_status: "published"
- confidentiality: "public"

2. Cách hiểu thủ công và dùng tool

Nhóm việc	Làm thủ công	Dùng tool	Cách tối ưu
Thu thập tài liệu	Chọn nguồn chính thống, xác định đúng đơn vị ban hành, ghi ngày truy cập.	Script tải file, lưu checksum, ghi log nguồn.	Thủ công xác nhận nguồn; tool tự động tải/lưu metadata.
OCR/Parser	Kiểm tra tài liệu là text PDF hay scan; đối chiếu vài trang quan trọng.	PyMuPDF/pdfplumber cho PDF text; PaddleOCR/EasyOCR/VietOCR cho scan.	Ưu tiên parser text trước; chỉ OCR trang ảnh để tiết kiệm thời gian.
Chuẩn hóa Markdown	Sửa heading, bảng, danh sách, page marker; không diễn giải lại nội dung.	Markdown formatter, regex cleanup, table checker.	Tool làm sạch 80%; người review phần pháp lý/nghiệp vụ.
Metadata/hiệu lực	Xác định số hiệu, ngày ban hành, hiệu lực, bản thay thế.	YAML validator, schema checker, checksum.	Tool kiểm tra thiếu/sai kiểu dữ liệu; người quyết định hiệu lực.
RAG ingest	Không làm thủ công từng chunk nếu nhiều tài liệu.	Backend chunking, embedding, Qdrant indexing.	Tự động hóa backend nhưng có bước preview và test truy vấn.

3. Phân tích từng bước chi tiết

Bước 0 - Nguyên tắc cốt lõi

Mục tiêu: Đặt luật kiểm soát để không đưa tài liệu sai, chưa duyệt hoặc hết hiệu lực vào chatbot.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Xác định 4 vùng dữ liệu: file gốc để audit, Markdown chuẩn để RAG, backend để validate/chunk/embed/index, Qdrant để lưu vector. Viết rõ rule publish và rule skip ngay từ đầu dự án.
Phần làm thủ công	Tự định nghĩa quy ước thư mục, trạng thái metadata, cách đặt tên tài liệu và tiêu chí tài liệu được phép vào RAG.
Phần dùng tool xử lý	Dùng YAML schema, script validate metadata, Git để quản lý thay đổi, Qdrant filter để chặn tài liệu không hợp lệ.
Công nghệ/công cụ nên dùng	Markdown, YAML, Git, Python validator, Qdrant payload filter.
Cách tối ưu nhất	Làm bộ rule cứng ngay từ đầu: unchecked thì not_indexed, replaced thì deactivated, chỉ approved + valid + public mới publish.

Bước 1 - Thu thập tài liệu gốc

Mục tiêu: Lấy đúng tài liệu từ nguồn chính thống hoặc nguồn nội bộ được phép sử dụng.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Tìm tài liệu trên website CTU/phòng ban, biểu mẫu, quyết định, quy định. Ghi source_url, source_file, accessed_date. Không chỉnh sửa nội dung file gốc.
Phần làm thủ công	Người làm cần kiểm tra nguồn có đáng tin hay không, tài liệu có đúng phạm vi đề tài không, có phải bản mới nhất không.
Phần dùng tool xử lý	Dùng script tải file từ URL, tự ghi tên file, ngày tải, checksum và log thu thập.
Công nghệ/công cụ nên dùng	Browser, Python requests, pathlib, hashlib, Git/Git LFS nếu file lớn.
Cách tối ưu nhất	Thu thập thủ công nguồn lần đầu, sau đó dùng script tải và ghi log để tránh nhầm file hoặc mất nguồn.

Bước 2 - Lưu file gốc vào 02_Attachments

Mục tiêu: Lưu file gốc đúng thư mục để làm nguồn đối chiếu/audit về sau.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Phân loại theo PDFs/DOCX/Forms/Images và theo đơn vị như PDT, CTSV. Đặt tên file có số hiệu, tên ngắn, năm. Kiểm tra file mở được.

Nội dung cần phân tích	Cách thực hiện cụ thể
Phần làm thủ công	Người làm quyết định file thuộc đơn vị/lĩnh vực nào và kiểm tra file có bị lỗi hoặc tải thiếu không.
Phần dùng tool xử lý	Script tự tạo folder, chuẩn hóa tên file, kiểm tra trùng tên, tính checksum.
Công nghệ/công cụ nên dùng	Python pathlib/shutil, hashlib, Git, quy tắc đặt tên file.
Cách tối ưu nhất	Không lưu file gốc trong 01_Dataset. 01_Dataset chỉ chứa Markdown chuẩn, còn 02_Attachments giữ file gốc.

Bước 3 - OCR / Parser ra Markdown tạm

Mục tiêu: Chuyển file gốc thành Markdown có cấu trúc ban đầu.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Nếu PDF bôi đen được thì parser text trực tiếp. Nếu PDF scan/ảnh thì OCR. Giữ heading, chương, điều, khoản, điểm, bảng, danh sách, số trang, link. Thêm page marker dạng <code><!-- page: n --></code> .
Phần làm thủ công	Người làm chọn hướng xử lý: parser hay OCR; kiểm tra vài trang đầu/cuối và các trang có bảng.
Phần dùng tool xử lý	Tool thực hiện trích text, OCR ảnh, nhận diện bảng, xuất file tạm vào 06_Processing/OCR_Output/<ten_file>_ocr.md.
Công nghệ/công cụ nên dùng	PyMuPDF, pdfplumber, PaddleOCR, EasyOCR, VietOCR, Tesseract, python-docx, markdownify, BeautifulSoup.
Cách tối ưu nhất	Tối ưu nhất là pipeline hybrid: thử PyMuPDF/pdfplumber trước; chỉ OCR trang không có text hoặc trang ảnh. Như vậy nhanh hơn OCR toàn bộ PDF.

Bước 4 - Lưu bản OCR tạm vào 06_Processing/OCR_Output

Mục tiêu: Giữ bản OCR thô/tạm để đối chiếu trước khi làm sạch.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Sau OCR/parser, lưu ra file _ocr.md. Không chỉnh trực tiếp file tạm quá nhiều; file tạm dùng để truy vết lỗi OCR.
Phần làm thủ công	Người làm đặt tên file đúng với file gốc và kiểm tra output có đủ số trang không.
Phần dùng tool xử lý	Script tự sinh output, thêm page marker, ghi parser, parser_version, ocr_engine, ocr_confidence nếu có.
Công nghệ/công cụ nên dùng	Python, pathlib, logging, Markdown, metadata log JSON/YAML.
Cách tối ưu nhất	Giữ riêng OCR_Output để có thể so sánh raw OCR với bản Markdown chuẩn trong 01_Dataset.

Bước 5 - Làm sạch Markdown

Mục tiêu: Biến bản OCR tạm thành Markdown chuẩn, dễ review, dễ chunking.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Sửa heading đúng cấp, nối dòng bị vỡ, xóa header/footer lặp, sửa bảng, danh sách, page marker, lỗi dấu tiếng Việt. Không được tự ý đổi nội dung nghiệp vụ.
Phần làm thủ công	Người làm phải đọc đối chiếu với PDF gốc, nhất là số điều, số khoản, tên phòng ban, ngày tháng, bảng biểu.
Phần dùng tool xử lý	Tool hỗ trợ regex cleanup, format bảng, phát hiện dòng lặp, kiểm tra page marker, so sánh số trang.
Công nghệ/công cụ nên dùng	Python regex, markdownlint, mdformat, pandas/tabulate cho bảng, diff tools.
Cách tối ưu nhất	Tool làm sạch định dạng trước, sau đó người review các đoạn quan trọng. Không để AI tự diễn giải lại văn bản pháp lý/học vụ.

Bước 6 - Tạo file Markdown chính trong 01_Dataset

Mục tiêu: Tạo canonical Markdown làm nguồn chính cho review, chunk, embedding và RAG.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Copy nội dung đã sạch sang 01_Dataset/<Đơn vị hoặc Lĩnh vực>/<ten_tai_lieu>.md. File này không phải raw OCR mà là bản chuẩn.
Phần làm thủ công	Người làm quyết định tên tài liệu, thư mục đúng đơn vị/lĩnh vực và đảm bảo không ghi đè phiên bản cũ.
Phần dùng tool xử lý	Script kiểm tra trùng tên, tạo file mới theo template, chèn YAML metadata đầu file.
Công nghệ/công cụ nên dùng	Markdown, YAML front matter, Git, Obsidian/VS Code.
Cách tối ưu nhất	Chỉ chunk và embedding từ 01_Dataset, không lấy trực tiếp từ OCR_Output.

Bước 7 - Điền YAML metadata

Mục tiêu: Gắn thông tin máy có thể đọc để backend validate, filter và citation.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Điền document_id, version_id, title, document_type, domain, department, audience, code, issued_date, effective_date, version, status, source_url, source_file, checksum, parser.
Phần làm thủ công	Người làm xác định thông tin nghiệp vụ: loại tài liệu, đơn vị, hiệu lực, bản thay thế, mức độ công khai.

Nội dung cần phân tích	Cách thực hiện cụ thể
Phần dùng tool xử lý	Tool validate schema, phát hiện thiếu trường, sai format ngày, sai enum trạng thái, tự tính checksum.
Công nghệ/công cụ nên dùng	YAML, Pydantic/JSON Schema, Python, pre-commit hook.
Cách tối ưu nhất	Dùng template metadata cố định. Không dùng một trường status chung chung; tách collection_status, ocr_status, review_status, rag_status, validity_status.

Bước 8 - Kiểm tra document_id và version_id

Mục tiêu: Đảm bảo quản lý được nhiều phiên bản của cùng một nhóm tài liệu.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	document_id là ID chung cho nhóm tài liệu; version_id là ID riêng cho từng bản. Bản mới không ghi đè bản cũ.
Phần làm thủ công	Người làm xác định tài liệu này cùng nhóm với tài liệu cũ hay là tài liệu hoàn toàn khác.
Phần dùng tool xử lý	Tool quét 01_Dataset để phát hiện version_id trùng, document_id bất thường, file bị overwrite.
Công nghệ/công cụ nên dùng	Python scanner, YAML parser, Git history, Dataview trong Obsidian.
Cách tối ưu nhất	Quy ước đặt ID ngay từ đầu, ví dụ document_id ổn định theo chủ đề, version_id chứa số quyết định/năm.

Bước 9 - Kiểm tra hiệu lực tài liệu

Mục tiêu: Xác định tài liệu còn hiệu lực, hết hạn, chưa rõ hay đã bị thay thế.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Đọc phần hiệu lực, ngày ban hành, văn bản thay thế, thông báo mới. Nếu chưa chắc thì để unchecked và không publish.
Phần làm thủ công	Đây là bước cần con người quyết định nhiều nhất vì liên quan nghiệp vụ và độ đúng của chatbot.
Phần dùng tool xử lý	Tool hỗ trợ tìm văn bản mới hơn, so sánh code/ngày, phát hiện replaced_by/replaces còn thiếu.
Công nghệ/công cụ nên dùng	Browser, search site CTU, Python metadata checker, Dataview query.
Cách tối ưu nhất	Không đoán hiệu lực. Nếu không chắc: validity_status = unchecked, rag_status = not_indexed.

Bước 10 - Review nội dung

Mục tiêu: Kiểm tra bản Markdown chuẩn có đúng với file gốc và đủ điều kiện dùng cho RAG không.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Review tiêu đề, số hiệu, ngày ban hành, đơn vị, đủ nội dung OCR, heading, bảng, page marker, thông tin nhạy cảm.
Phần làm thủ công	Người review phải đọc các phần quan trọng và đối chiếu file gốc, nhất là bảng và điều khoản.
Phần dùng tool xử lý	Tool hỗ trợ diff, checklist, phát hiện thiếu page marker, bảng vỡ, OCR confidence thấp.
Công nghệ/công cụ nên dùng	VS Code diff, markdown preview, Python QA script, checklist Markdown.
Cách tối ưu nhất	Nên có 2 lớp: người format tự kiểm trước, người review duyệt sau. Chỉ approved khi đã đối chiếu nguồn.

Bước 11 - Quyết định publish vào RAG

Mục tiêu: Chỉ cho tài liệu đạt chuẩn đi vào pipeline RAG.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Nếu ocr_status done, review_status approved, validity_status valid, rag_status published, confidentiality public thì được publish. Nếu dùng rule nghiêm ngặt, thêm is_latest true.
Phần làm thủ công	Người làm quyết định đổi trạng thái rag_status từ not_indexed sang published sau khi đủ điều kiện.
Phần dùng tool xử lý	Tool validate điều kiện publish, chặn publish nếu thiếu trường hoặc sai trạng thái.
Công nghệ/công cụ nên dùng	Python validator, Pydantic, Git pre-commit/CI, YAML parser.
Cách tối ưu nhất	Không publish bằng cảm tính. Dùng script validate pass mới cho copy hoặc ingest.

Bước 12 - Copy sang 04_RAG/Published nếu dùng MVP workflow

Mục tiêu: Tạo vùng chứa riêng cho tài liệu đã duyệt, giúp demo và kiểm soát dễ hơn.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Cách A: backend đọc trực tiếp 01_Dataset. Cách B: copy file approved sang 04_RAG/Published. Với MVP nên dùng cách B.
Phần làm thủ công	Người làm xác nhận file nào được publish và khi nào copy.
Phần dùng tool xử lý	Tool copy tự động các file đã pass validate, đồng thời tạo manifest published_files.json.
Công nghệ/công cụ nên dùng	Python shutil, YAML validator, Git, folder 04_RAG/Published.
Cách tối ưu nhất	MVP nên dùng Cách B để tránh backend ingest nhầm file đang sửa trong 01_Dataset.

Bước 13 - Backend validate metadata

Mục tiêu: Backend kiểm tra lần cuối trước khi ingest để đảm bảo an toàn dữ liệu.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Backend đọc YAML, kiểm tra các trường bắt buộc, trạng thái, effective_date, expiry_date, confidentiality, is_latest nếu cần.
Phân làm thủ công	Người làm định nghĩa rule backend và test các trường hợp pass/fail.
Phần dùng tool xử lý	Tool/backend tự động skip file không hợp lệ và ghi log lý do skip.
Công nghệ/công cụ nên dùng	FastAPI hoặc Python service, Pydantic, YAML parser, logging, unit test.
Cách tối ưu nhất	Validate hai lớp: trước khi publish và ngay trong backend ingest. Không tin hoàn toàn vào thao tác thủ công.

Bước 14 - Chunking từ Markdown

Mục tiêu: Chia Markdown thành các đoạn nhỏ có nghĩa để truy xuất chính xác.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Chunk theo heading path như Tài liệu > Chương > Điều > Khoản; giữ page_start, page_end và metadata cho mỗi chunk.
Phân làm thủ công	Người làm quyết định chiến lược chunk phù hợp với tài liệu pháp lý/học vụ.
Phần dùng tool xử lý	Tool chunk tự động theo heading, giới hạn độ dài, overlap nhẹ nếu cần, xuất preview để kiểm tra.
Công nghệ/công cụ nên dùng	Python Markdown parser, markdown-it-py, mistune, LangChain/LlamaIndex text splitter tùy chỉnh.
Cách tối ưu nhất	Không cắt ngẫu nhiên theo số ký tự trước. Ưu tiên heading-based để giữ ngữ cảnh Điều/Khoản/Điểm.

Bước 15 - Embedding

Mục tiêu: Biến chunk thành vector để tìm kiếm ngữ nghĩa.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Input embedding nên gồm title, heading path, metadata context ngắn và chunk content. Không embed file chưa approved/valid/published.
Phân làm thủ công	Người làm chọn model embedding phù hợp tiếng Việt và chi phí.
Phần dùng tool xử lý	Tool/backend gọi model embedding, batch nhiều chunk,

Nội dung cần phân tích	Cách thực hiện cụ thể
	cache theo checksum để tránh embed lại không cần thiết.
Công nghệ/công cụ nên dùng	OpenAI embeddings hoặc bge-m3/multilingual-e5, sentence-transformers, batching, cache SQLite/JSON.
Cách tối ưu nhất	Embed kèm heading path để câu trả lời hiểu ngữ cảnh. Cache vector theo chunk_id + checksum để tiết kiệm thời gian.

Bước 16 - Qdrant indexing

Mục tiêu: Lưu vector và payload vào Qdrant để retriever tìm kiếm.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	Mỗi point cần chunk_id, document_id, version_id, title, document_type, domain, department, audience, is_latest, validity_status, rag_status, page_start/page_end, source_url/source_file.
Phần làm thủ công	Người làm thiết kế collection, payload filter và rule truy vấn.
Phần dùng tool xử lý	Tool/backend upsert vector vào Qdrant, update payload khi tài liệu replaced/deactivated, test truy vấn.
Công nghệ/công cụ nên dùng	Qdrant, qdrant-client, FastAPI, Docker, payload index, vector search.
Cách tối ưu nhất	Retriever mặc định filter: rag_status=published, validity_status=valid, confidentiality=public. Có thể thêm is_latest=true để nghiêm ngặt.

Bước 17 - Khi tài liệu bị thay thế

Mục tiêu: Cập nhật trạng thái để chatbot không trả lời theo bản cũ.

Nội dung cần phân tích	Cách thực hiện cụ thể
Việc cần làm	File cũ đặt is_latest false, validity_status replaced, replaced_by version mới, rag_status deactivated. File mới đặt is_latest true, valid, replaces version cũ, approved, published.
Phần làm thủ công	Người làm xác nhận quan hệ thay thế giữa bản cũ và mới.
Phần dùng tool xử lý	Tool cập nhật metadata hàng loạt và update payload vector cũ trong Qdrant.
Công nghệ/công cụ nên dùng	Python metadata updater, Qdrant payload update, Dataview kiểm tra replaced chưa deactivate.
Cách tối ưu nhất	Không cần xóa vector cũ ngay; chỉ cần deactivated và retriever filter bỏ qua. Cách này vẫn giữ lịch sử/audit.

4. Workflow tối ưu đề xuất cho đồ án NLCS RAG

Giai đoạn	Việc làm	Cách tối ưu
Giai đoạn 1 - Chuẩn bị dữ liệu	Thu thập file gốc, lưu vào 02_Attachments, ghi source_url/accessed_date/checksum.	Thủ công chọn nguồn; tool tải và lưu log.
Giai đoạn 2 - OCR/Parser	Chạy parser text trước, OCR chỉ khi PDF/ảnh không có text. Xuất _ocr.md vào 06_Processing/OCR_Output.	Tool xử lý chính; người kiểm tra output.
Giai đoạn 3 - Chuẩn hóa	Làm sạch Markdown, sửa heading/bảng/page marker, tạo file canonical trong 01_Dataset.	Tool cleanup + người review.
Giai đoạn 4 - Metadata/Review	Điền YAML, kiểm tra version_id, hiệu lực, review_status.	Người quyết định hiệu lực; tool validate schema.
Giai đoạn 5 - Publish/Ingest	Copy sang 04_RAG/Published, backend validate, chunk, embed, index Qdrant.	Tool/backend tự động; người test câu hỏi.

5. Công nghệ khuyến nghị theo tầng

Tầng xử lý	Công nghệ	Lý do dùng
Thu thập/lưu file	Python requests, pathlib, hashlib, Git/Git LFS	Tự động tải file, tạo checksum, tránh thất lạc nguồn.
PDF text parser	PyMuPDF, pdfplumber	Nhanh hơn OCR, giữ được text nếu PDF bôi đen được.
OCR ảnh/scan	PaddleOCR, EasyOCR, VietOCR, Tesseract	Dùng khi PDF là ảnh hoặc tài liệu scan; ưu tiên engine hỗ trợ tiếng Việt tốt.
Markdown cleanup	Python regex, mdformat, markdownlint, pandas/tabulate	Chuẩn hóa heading, list, bảng, page marker.
Metadata validation	YAML, Pydantic, JSON Schema, pre-commit	Chặn thiếu trường, sai trạng thái, sai format ngày.
Backend RAG	FastAPI, Python, LangChain/LlamaIndex tùy nhu cầu	Validate, chunk, embedding, truy vấn RAG.
Embedding	OpenAI embeddings, bge-m3, multilingual-e5	Cần model hỗ trợ tiếng Việt tốt.
Vector DB	Qdrant, qdrant-client, Docker	Lưu vector và payload metadata để filter.
Review/quản lý tri thức	VS Code, Obsidian, Dataview, Git	Dễ đọc Markdown, query metadata, quản lý version.

6. Checklist triển khai nhanh

- File gốc đã nằm đúng trong 02_Attachments, không nằm trong 01_Dataset.
 - Bản OCR/parser tạm đã lưu vào 06_Processing/OCR_Output.
 - File Markdown chính đã nằm trong 01_Dataset và có YAML metadata.
- Phân tích Quick Guide - Quy trình xử lý tài liệu cho NLCS RAG

- document_id ổn định theo nhóm tài liệu; version_id riêng cho từng phiên bản.
- validity_status chỉ để valid khi đã kiểm tra chắc chắn hiệu lực.
- review_status chỉ để approved khi đã đối chiếu với file gốc.
- Tài liệu unchecked/unknown/expired/replaced không được đưa vào RAG.
- File published phải pass backend validate trước khi chunk/embedding.
- Chunk phải có heading_path và page_start/page_end để citation.
- Qdrant retriever phải filter rag_status=published, validity_status=valid, confidentiality=public.
- Test truy vấn thử phải trả lời đúng nội dung và trích dẫn đúng trang.

7. Kết luận ngắn gọn

Cách triển khai tối ưu là kết hợp thủ công và tự động hóa: con người chịu trách nhiệm xác nhận nguồn, hiệu lực, nội dung nghiệp vụ và quyết định approved/published; tool chịu trách nhiệm OCR/parser, làm sạch định dạng, validate metadata, chunking, embedding và indexing. Với MVP, nên dùng thư mục 04_RAG/Published làm vùng đệm để backend chỉ ingest những file đã duyệt rõ ràng.

File gốc → OCR/Parser → Markdown chuẩn → Metadata + Review → Approved + Valid → Published → Chunking → Embedding → Qdrant → Active Knowledge Base